

DOCUMENT 3

Data Handling & Preprocessing

The Complete One-Pass Refresher Guide

AI Engineer Placement Preparation Series

Generated: February 2026 | Cutting-Edge Edition

Table of Contents

Why This Matters

Data scientists and ML engineers spend 60-80% of their time on data preparation, not model building. The most sophisticated model in the world will produce garbage if fed garbage data. Interviewers know this. They test data handling skills because they're the most practically relevant skills you'll use on the job.

For AI Engineers specifically: the rise of RAG systems and multimodal AI means you now work with PDFs, images, and audio, not just clean CSV files. Document chunking, embedding generation, and unstructured data parsing are now core skills, not niche specialties.

2025-2026 Shift

Interviewers increasingly give you a messy dataset and ask you to clean it live. They also ask about chunking strategies for RAG pipelines, which is data preprocessing applied to LLM systems. If you only know tabular data cleaning, you're behind.

1. NumPy

NumPy is the foundation everything else is built on. Pandas uses NumPy internally. PyTorch tensors are conceptually identical to NumPy arrays. If you understand NumPy, you understand the data layer of all ML.

1.1 ndarray: The Core Object

ndarray (N-dimensional array) is a fixed-size, homogeneous (all same type) grid of numbers stored in contiguous memory. This is why it's 10-100x faster than Python lists for numerical operations.

```
import numpy as np

a = np.array([1, 2, 3])           # 1D array (vector)
b = np.array([[1,2],[3,4]])      # 2D array (matrix)
c = np.zeros((3, 4))            # 3x4 matrix of zeros
d = np.random.randn(2, 3)       # 2x3 matrix, random normal
e = np.arange(0, 10, 2)         # [0, 2, 4, 6, 8]
f = np.linspace(0, 1, 5)       # [0, 0.25, 0.5, 0.75, 1.0]
```

1.2 Indexing, Slicing, Reshaping

```
arr = np.array([[1,2,3],[4,5,6],[7,8,9]])

arr[0, 1]          # 2 (row 0, col 1)
arr[:, 1]          # [2, 5, 8] (all rows, col 1)
arr[1:, :2]        # [[4,5],[7,8]] (rows 1+, cols 0-1)
arr[arr > 5]       # [6, 7, 8, 9] (boolean indexing)

arr.reshape(1, 9)  # [[1,2,3,4,5,6,7,8,9]] (reshape to 1x9)
arr.reshape(-1)    # [1,2,...,9] (flatten; -1 = infer size)
arr.T              # transpose
```

1.3 Broadcasting

Broadcasting is NumPy's way of performing operations between arrays of different shapes without explicit loops. It automatically "stretches" the smaller array to match the larger one.

```
a = np.array([[1,2,3],[4,5,6]]) # shape (2,3)
b = np.array([10, 20, 30])      # shape (3,)
a + b                           # shape (2,3) - b is broadcast across rows
# Result: [[11,22,33],[14,25,36]]

# Rule: Shapes are compared right-to-left.
# Dimensions match if they're equal OR one of them is 1.
```

AI relevance: Broadcasting is how batch operations work. When you add a bias vector to a batch of inputs, you're broadcasting $[1 \times n_features]$ across $[batch_size \times n_features]$.

1.4 Key Operations

Operation	Code	What It Does
Matrix multiply	<code>np.dot(A, B)</code> or <code>A @ B</code>	Standard matrix multiplication
Element-wise multiply	<code>A * B</code>	Multiply corresponding elements
Sum/Mean/Std	<code>np.sum(a, axis=0)</code>	Aggregate along axis (0=rows, 1=cols)
Concatenate	<code>np.concatenate([a,b], axis=0)</code>	Stack arrays along an axis
Where (conditional)	<code>np.where(a > 0, a, 0)</code>	Element-wise if/else (like ReLU!)
Argmax/Argmin	<code>np.argmax(a, axis=1)</code>	Index of max/min along axis
Unique	<code>np.unique(a, return_counts=True)</code>	Unique values + their counts

2. Pandas

Pandas is the Swiss Army knife for tabular data. If your data fits in memory and has rows and columns, Pandas is your tool.

2.1 Series and DataFrame

Series = a single column (1D labeled array). **DataFrame** = a table (2D labeled structure, like a spreadsheet or SQL table). Every DataFrame column is a Series.

```
import pandas as pd

df = pd.read_csv('data.csv')          # Load CSV
df = pd.read_json('data.json')        # Load JSON
df = pd.read_parquet('data.parquet')  # Load Parquet (faster, smaller)

df.head()          # First 5 rows
df.shape           # (rows, cols) - NOTE: attribute, no parentheses
df.info()          # Column types, non-null counts, memory usage
df.describe()      # Statistical summary (mean, std, min, max, quartiles)
df.dtypes          # Data type of each column
```

2.2 Indexing: loc vs iloc

Method	Syntax	Type	Example
loc	df.loc[row_label, col_label]	Label-based	df.loc[0:5, 'name'] (includes row 5)
iloc	df.iloc[row_idx, col_idx]	Integer-based	df.iloc[0:5, 0] (excludes row 5)
Boolean	df[df['age'] > 25]	Condition	Filters rows where age > 25
Multiple cols	df[['name', 'age']]	List of names	Select specific columns

2.3 Essential Operations

```
# Groupby: Split-Apply-Combine
df.groupby('category')['sales'].mean()    # Mean sales per category
df.groupby(['city', 'year']).agg({'sales': 'sum', 'profit': 'mean'})

# Merge (SQL-style joins)
pd.merge(df1, df2, on='id', how='left')   # left/right/inner/outer

# Apply: custom functions
df['new_col'] = df['text'].apply(lambda x: len(x.split()))

# Pivot table
df.pivot_table(values='sales', index='region', columns='quarter', aggfunc='sum')
```

# Value counts <code>df['category'].value_counts()</code>	# Frequency of each category
--	------------------------------

2.4 Performance Tips (Interview Edge)

Tip	Why	How
Use PyArrow backend	2-5x faster reads, better memory	<code>pd.read_csv('f.csv', engine='pyarrow')</code>
Vectorize, never loop	Loops are 100x slower than vectorized ops	Use <code>.apply()</code> or built-in methods, never for-loops on rows
Use category dtype	Strings waste memory for repeated values	<code>df['col'] = df['col'].astype('category')</code>
Load only needed cols	Reduces memory by 50%+	<code>pd.read_csv('f.csv', usecols=['a','b'])</code>
Use Parquet over CSV	Binary format, 5-10x faster, smaller files	<code>df.to_parquet('data.parquet')</code>

3. Data Cleaning

Real-world data is messy. Missing values, duplicates, wrong types, outliers. This section covers the standard cleaning pipeline every AI engineer must know.

3.1 Handling Missing Data

Strategy	Code	When to Use
Drop rows	<code>df.dropna()</code>	Very few missing values (<5%), random missingness
Drop columns	<code>df.drop(columns=['col'])</code>	Column has >50% missing, not important
Fill with mean/median	<code>df['col'].fillna(df['col'].median())</code>	Numerical, roughly symmetric distribution
Fill with mode	<code>df['col'].fillna(df['col'].mode()[0])</code>	Categorical features
Forward/backward fill	<code>df['col'].fillna(method='ffill')</code>	Time-series data (carry last known value)
Interpolation	<code>df['col'].interpolate(method='linear')</code>	Time-series, smooth progression expected
KNN Imputation	<code>KNNImputer(n_neighbors=5).fit_transform(df)</code>	Complex relationships between features

Interview Trap

"When should you NOT fill missing values?" Answer: When missingness itself is informative. Example: a missing income field might indicate unemployment. In that case, create a binary indicator column (`is_missing`) instead of imputing.

3.2 Handling Duplicates

```
df.duplicated().sum()      # Count duplicates
df.drop_duplicates(inplace=True)  # Remove exact duplicates
df.drop_duplicates(subset=['id'])  # Remove based on specific column
df.drop_duplicates(keep='last')    # Keep last occurrence
```

3.3 Outlier Detection

Method	Logic	Code Pattern
Z-Score	Flag if $ z > 3$ (3 std devs from mean)	<code>from scipy import stats; np.abs(stats.zscore(df['col'])) > 3</code>
IQR	Flag if below $Q1 - 1.5 * IQR$ or above $Q3 + 1.5 * IQR$	<code>Q1, Q3 = df['col'].quantile([.25, .75]); IQR = Q3 - Q1</code>
Domain	Use business rules	<code>df[df['age'] < 0] # Age can't be negative</code>

Method	Logic	Code Pattern
knowledge		

What to do with outliers: Remove them (if errors), cap them (winsorization), transform them (log), or leave them (if they're real and informative). Context matters.

3.4 Data Type Conversions

```
df['price'] = pd.to_numeric(df['price'], errors='coerce') # String to number
df['date'] = pd.to_datetime(df['date'])                  # String to datetime
df['category'] = df['category'].astype('category')       # String to category
df['flag'] = df['flag'].astype(bool)                     # To boolean
```

4. Feature Engineering

Feature engineering is the art of creating inputs that help your model learn better. Good features can make a simple model outperform a complex one with bad features.

4.1 Encoding Categorical Variables

Method	How	When to Use	Watch Out For
Label Encoding	Map categories to integers (A=0, B=1, C=2)	Ordinal data (low/med/high)	Don't use for nominal data! Model assumes $2 > 1 > 0$.
One-Hot Encoding	Create binary column per category	Nominal data with few categories (<10-15)	High cardinality = too many columns (curse of dim.)
Target Encoding	Replace category with mean of target variable	High cardinality categorical features	Risk of data leakage! Must compute on train set only.
Frequency Encoding	Replace category with its count/frequency	When frequency is informative	Ties: different categories with same frequency get same value.

4.2 Feature Scaling

Method	Formula	Range	When to Use
Normalization (Min-Max)	$(x - \min) / (\max - \min)$	[0, 1]	Neural networks, KNN, algorithms sensitive to range
Standardization (Z-score)	$(x - \text{mean}) / \text{std}$	$\sim [-3, 3]$	Linear regression, SVM, PCA, most algorithms
Robust Scaling	$(x - \text{median}) / \text{IQR}$	varies	Data with outliers (uses median, not mean)
Log Transform	$\log(x + 1)$	varies	Right-skewed data (income, prices, counts)

Critical Interview Point: `fit_transform()` vs `transform()`

ALWAYS fit on training data, then transform both train and test. Never fit on test data! This is data leakage. Use `scaler.fit_transform(X_train)` then `scaler.transform(X_test)`. If you fit on test data, you're using future information to transform your data.

4.3 Date/Time Features

```
df['date'] = pd.to_datetime(df['date'])
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
```

```
df['day_of_week'] = df['date'].dt.dayofweek # 0=Monday
df['is_weekend'] = df['day_of_week'].isin([5,6]).astype(int)
df['hour'] = df['date'].dt.hour
df['days_since'] = (pd.Timestamp.now() - df['date']).dt.days
```

4.4 Text Features (For NLP Pipelines)

```
df['word_count'] = df['text'].str.split().str.len()
df['char_count'] = df['text'].str.len()
df['has_url'] = df['text'].str.contains(r'https?://').astype(int)
df['upper_ratio'] = df['text'].apply(lambda x: sum(c.isupper() for c in x)/len(x))
```

4.5 Feature Selection

Method	Type	How It Works	Code
Correlation filter	Filter	Drop features with >0.95 correlation to another	df.corr() then drop one of the pair
Variance threshold	Filter	Drop features with near-zero variance	VarianceThreshold(threshold=0.01)
Mutual information	Filter	Score features by info gained about target	mutual_info_classif(X, y)
RFE	Wrapper	Train model, remove weakest feature, repeat	RFE(estimator, n_features_to_select=10)
L1 Regularization	Embedded	Lasso drives irrelevant weights to zero	LogisticRegression(penalty='l1')
Tree importance	Embedded	Random Forest ranks features by split importance	model.feature_importances_

5. Working with Unstructured Data

This section gives you a cutting-edge advantage. Most data prep guides only cover tabular data. But in 2025-2026, AI Engineers work with PDFs, images, audio, and long documents for RAG systems.

5.1 Document Chunking (For RAG Systems)

When building RAG pipelines, you split documents into chunks before embedding them. The chunking strategy directly affects retrieval quality.

Strategy	How	Pros	Cons
Fixed-size	Split every N characters/tokens	Simple, predictable	Breaks sentences mid-thought
Sentence-based	Split at sentence boundaries	Respects natural units	Sentences vary wildly in length
Semantic	Group by topic using embeddings	Best retrieval quality	Expensive, slower to process
Recursive	Split by paragraphs, then sentences if too long	Good balance	Slightly complex implementation
Overlapping	Each chunk overlaps with previous by N tokens	Preserves context at boundaries	More chunks = more storage/cost

Typical chunk sizes: For text embedding models, 256-512 tokens works well. Larger chunks (1000+) capture more context but reduce retrieval precision. Smaller chunks (<128) are precise but may lose context.

5.2 Embedding Generation

Embeddings convert text (or images, audio) into dense numerical vectors that capture semantic meaning.

```
# Using sentence-transformers (most common for RAG)
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(['Hello world', 'Hi there']) # shape: (2, 384)

# Using OpenAI API
# response = client.embeddings.create(
#     model='text-embedding-3-small', input=['Hello world']
# )
```

5.3 Working with PDFs and Images

Data Type	Common Tools	Preprocessing Steps
PDFs	PyPDF2, pdfplumber, unstructured	Extract text, handle tables, OCR for scanned pages
Images	Pillow (PIL), OpenCV, torchvision	Resize, normalize pixels to [0,1], augment (flip, rotate)
Audio	librosa, whisper	Convert to spectrogram or transcribe to text, resample
HTML/Web	BeautifulSoup, trafilatura	Strip tags, extract main content, handle encoding

6. Data Visualization

Visualization is your tool for understanding data before modeling, and for communicating results after. Know when to use which chart.

Question You're Answering	Chart Type	Library Code
What's the distribution of one variable?	Histogram / KDE	<code>sns.histplot(df['col']) / sns.kdeplot()</code>
How do two variables relate?	Scatter plot	<code>plt.scatter(df['x'], df['y'])</code>
What are the correlations between features?	Heatmap	<code>sns.heatmap(df.corr(), annot=True)</code>
How do categories compare?	Bar chart	<code>sns.barplot(x='category', y='value', data=df)</code>
What's the spread within categories?	Box plot	<code>sns.boxplot(x='category', y='value', data=df)</code>
How does a variable change over time?	Line chart	<code>plt.plot(df['date'], df['value'])</code>
How do all features relate to each other?	Pair plot	<code>sns.pairplot(df, hue='target')</code>
What's the distribution shape + outliers?	Violin plot	<code>sns.violinplot(x='cat', y='val', data=df)</code>

7. Scikit-Learn Pipelines

Pipelines chain preprocessing and modeling steps into a single object. This prevents data leakage, makes code cleaner, and is the professional way to build ML workflows.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier

# Define preprocessing for different column types
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])

# Combine into a single preprocessor
preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_transformer, ['age', 'income', 'score']),
    ('cat', categorical_transformer, ['city', 'gender'])
])

# Full pipeline: preprocessing + model
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', RandomForestClassifier())
])

pipeline.fit(X_train, y_train) # Fits everything in order
pipeline.predict(X_test)      # Transforms then predicts
```

Why this matters: The pipeline ensures that `fit()` is only called on training data and `transform()` is called on both. This eliminates the most common source of data leakage in ML projects. Interviewers love asking about this.

8. Common Mistakes

Data leakage via fit_transform on test set: ALWAYS fit scaler/encoder on train set only, then transform test set. Using fit_transform on the full dataset leaks test info into training.

Using mean imputation for skewed data: Mean is sensitive to outliers. For skewed distributions (income, prices), use median instead.

One-hot encoding high-cardinality features: A feature with 10,000 unique values creates 10,000 columns. Use target encoding or embedding instead.

Dropping missing values without checking why: Missing data might be informative (MNAR). Always check if missingness correlates with the target.

Not handling unseen categories in test data: If test data has a category your encoder never saw, it crashes. Use handle_unknown='ignore' in OneHotEncoder.

Scaling the target variable in regression: You typically scale features, not the target. Scaling the target complicates interpretation.

Applying the same chunking strategy to all document types: Code documentation needs smaller chunks than narrative text. Always consider the document type.

Using .apply() with complex lambda when vectorized operations exist: df['col'].apply(lambda x: x*2) is 10x slower than df['col'] * 2. Use vectorized operations.

9. Quick-Fire Q&A

20 frequently asked questions with concise answers.

[Easy] Q: What's the difference between loc and iloc?

A: loc uses labels (inclusive), iloc uses integer positions (exclusive end). loc[0:5] returns 6 rows, iloc[0:5] returns 5.

[Easy] Q: How do you handle missing data?

A: Depends on context: drop if few and random, impute with median (numeric) or mode (categorical), use KNN imputer for complex patterns, or create an is_missing indicator.

[Easy] Q: What's the difference between normalization and standardization?

A: Normalization scales to [0,1] using min-max. Standardization centers to mean=0, std=1. Use normalization for bounded algorithms (neural nets), standardization for most others.

[Easy] Q: Explain one-hot encoding vs label encoding.

A: One-hot creates binary columns per category (no ordering assumed). Label encoding assigns integers (implies ordering). Use label for ordinal data, one-hot for nominal.

[Medium] Q: What is data leakage and how do you prevent it?

A: Using test/future information during training. Prevent: split data BEFORE any preprocessing, use pipelines, never fit on test data, be careful with time-series.

[Easy] Q: What is feature scaling and why is it important?

A: Bringing features to similar ranges. Important because gradient descent converges faster, distance-based algorithms (KNN, SVM) aren't dominated by large-scale features.

[Medium] Q: How does broadcasting work in NumPy?

A: Arrays of different shapes can operate together if their dimensions match right-to-left, with 1-sized dimensions expanded. [3,4] + [4] works, [3,4] + [3] doesn't.

[Medium] Q: What is a Pandas Pipeline and why use it?

A: Chains preprocessing + model steps. Prevents data leakage by ensuring fit only on train data. Makes code reproducible and deployable.

[Medium] Q: How would you handle a categorical feature with 50,000 unique values?

A: Target encoding, frequency encoding, or hash encoding. Never one-hot encode (creates 50K columns). Could also use embeddings if building a neural network.

[Easy] Q: What's the difference between merge and concat in Pandas?

A: merge joins on columns (like SQL JOIN). concat stacks DataFrames vertically (axis=0) or horizontally (axis=1). Merge matches rows, concat appends.

[Hard] Q: What chunking strategy would you use for a RAG system?

A: Recursive chunking with overlap. Start with paragraphs, split further if too long, keep 10-20% overlap. Typical size: 256-512 tokens. Adjust based on embedding model's context window.

[Easy] Q: How do you detect outliers?

A: Z-score (flag if $|z| > 3$), IQR method (flag if outside $Q1 - 1.5 \cdot IQR$ or $Q3 + 1.5 \cdot IQR$), or domain knowledge rules. Visualization (boxplot) is also essential.

[Medium] Q: What is target encoding?

A: Replace each category with the mean of the target for that category. E.g., if city='NYC' has avg income 85K, replace 'NYC' with 85000. Must only compute on training data.

[Medium] Q: Why use Parquet over CSV?

A: Parquet is columnar (read only needed columns), binary (5-10x smaller), faster to read, preserves data types. CSV is text-based and slow for large data.

[Medium] Q: What is the curse of dimensionality?

A: As features increase, data becomes sparse. Distance metrics become meaningless, models need exponentially more data. Solution: feature selection, PCA, embeddings.

[Medium] Q: How do you handle imbalanced classes in preprocessing?

A: Oversampling minority class (SMOTE), undersampling majority class, class weights in the model, or collecting more minority data. Choose based on dataset size.

[Easy] Q: What's the difference between fit(), transform(), and fit_transform()?

A: fit() learns parameters from data (mean, std). transform() applies learned parameters. fit_transform() does both in one step. Only use fit_transform on training data.

[Hard] Q: How do you generate embeddings for a RAG pipeline?

A: Use sentence-transformers (local) or OpenAI/Cohere API. Encode each chunk into a dense vector. Store in a vector database (FAISS, ChromaDB, Pinecone). Retrieve via cosine similarity.

[Medium] Q: What is a ColumnTransformer?

A: Applies different preprocessing to different columns. Numeric columns get scaling, categorical get encoding, text gets vectorization. All in one step, no data leakage.

[Medium] Q: How do you handle text data for a traditional ML model?

A: Tokenize, lowercase, remove stopwords, then vectorize using TF-IDF or CountVectorizer. For deep learning, use pretrained embeddings (Word2Vec, BERT) instead.

10. One-Line Cheat Sheet

Entire document compressed. One concept per line.

NumPy ndarray — Contiguous memory, homogeneous, 10-100x faster than Python lists.

Broadcasting — Arrays of different shapes auto-expand. Dimensions match right-to-left.

Pandas DataFrame — 2D labeled table. Every column is a Series. The core data structure.

loc vs iloc — loc = label-based (inclusive). iloc = integer-based (exclusive end).

groupby — Split data by category, apply function, combine results. The SQL GROUP BY.

merge — SQL-style joins. Inner/left/right/outer. Match on key columns.

Missing data — Drop, impute (mean/median/mode/KNN), or create indicator column.

Outlier detection — Z-score ($|z| > 3$), IQR ($1.5 \times \text{IQR}$ rule), or domain knowledge.

One-Hot Encoding — Binary column per category. For nominal data with few unique values.

Label Encoding — Integer per category. Only for ordinal data (low < medium < high).

Target Encoding — Replace category with target mean. High cardinality. Watch for leakage.

Normalization — $(x - \min) / (\max - \min)$. Scales to [0,1]. For neural nets, bounded algorithms.

Standardization — $(x - \text{mean}) / \text{std}$. Centers at 0. For most ML algorithms.

Log Transform — $\log(x+1)$. Compresses right-skewed data (income, prices).

Feature Selection — Correlation filter, variance threshold, mutual info, RFE, L1 penalty, tree importance.

Data Leakage — Using test info during training. Prevent with pipelines and proper splits.

Pipeline — Chain preprocessing + model. Prevents leakage, ensures reproducibility.

ColumnTransformer — Apply different transforms to different column types in one step.

Document Chunking — Split docs for RAG. Recursive + overlap is the default good choice.

Embeddings — Dense vector representations of text/images. Foundation of semantic search.

Parquet — Columnar binary format. 5-10x faster than CSV. Preserves types.

PyArrow backend — Pandas 2.0+ feature. Faster, more memory-efficient than NumPy backend.

Visualization — Histogram for distribution, scatter for relationships, heatmap for correlations.